Code:

```
import random
import math

N = 8

def heuristic(board):
    conflicts = 0
    for i in range(N):
        for j in range(i + 1, N):
            if board[i] == board[j] or abs(board[i] - board[j]) == abs(i - j):
                conflicts += 1
    return conflicts

def print_board(board):
    for row in range(N):
        print(" ".join("Q" if board[col] == row else "." for col in range(N)))
    print()

def get_neighbor(board):
    new_board = board[:]
    col = random.randint(0, N - 1)
    row = random.randint(0, N - 1)

    while row == new_board[col]:
        row = random.randint(0, N - 1)

    new_board[col] = row
    return new_board

def simulated_annealing():
    current = [random.randint(0, N - 1) for _ in range(N)]
    temperature = 100.0
    cooling_rate = 0.95
    step = 1

    print("Initial Board:")
    print_board(current)

    while temperature > 0.1:
        current_cost = heuristic(current)
```

```
        if current_cost == 0:
            print("Solution Found!")
            return current

        neighbor = get_neighbor(current)
        neighbor_cost = heuristic(neighbor)
        delta = neighbor_cost - current_cost
        probability = math.exp(-delta / temperature) if delta > 0 else 1.0

        print("Step:", step)
        print("Temperature:", round(temperature, 3))
        print("Current Conflicts:", current_cost)
        print("Neighbor Conflicts:", neighbor_cost)
        print("Acceptance Probability:", round(probability, 4))

        if delta <= 0 or random.random() < probability:
            current = neighbor
            print("Move Accepted")
        else:
            print("Move Rejected")

        print("Current Board:")
        print_board(current)

        temperature *= cooling_rate
        step += 1

    if heuristic(current) == 0:
        return current

    return None

solution = simulated_annealing()

if solution is not None:
    print("Final Solution:", [x + 1 for x in solution])
    print("Final Conflicts:", heuristic(solution))
    print("Final Chessboard:")
    print_board(solution)
else:
    print("No solution found. Run the program again.")
```

Output:

```
Initial Board:
. . . . . .
. . . . . Q .
Q . . . . . Q
. . . . . .
. Q Q . . Q . .
. . . Q . . . .
. . . . Q . . .
. . . . . .

Step: 1
Temperature: 100.0
Current Conflicts: 12
Neighbor Conflicts: 14
Acceptance Probability: 0.9802
Move Accepted
Current Board:
. . . . . .
```

```
Temperature: 100.0
Current Conflicts: 12
Neighbor Conflicts: 14
Acceptance Probability: 0.9802
Move Accepted
Current Board:
. . . . . .
. . . . . Q .
Q . . . . . Q
. Q . . . . .
. . Q . . Q . .
. . . Q . . . .
. . . . Q . . .
. . . . . .

Step: 2
Temperature: 95.0
Current Conflicts: 14
Neighbor Conflicts: 14
```

```
. . . . . . . Q
Q . . . . . . .
. . . Q . . . .
. Q . . . . . .
. . . . . . Q .

Solution Found!
Final Solution: [5, 7, 2, 6, 3, 1, 8, 4]
Final Conflicts: 0
Final Chessboard:
. . . . . Q . .
. . Q . . . . .
. . . . Q . . .
. . . . . . . Q
Q . . . . . . .
. . . Q . . . .
. Q . . . . . .
. . . . . . Q .
```